ExRL: Only “How Long” Sample-Eff. Online Long-Chunk RL

ExRL: Sample-Efficient Online Adaptation of Long-Chunk Policies via Execution Length Control

Xu Luo, Jiaying Yang, Hao Wu, Junlin Xie, Zijin Hong, Ji Zhang, Lianli Gao, Jingkuan Song
UESTC • Tongji University • The Hong Kong Polytechnic University

arXiv 2611.13482 • First submitted 2026-11-10 • exrl-project.github.io • Paper Review

One-Sentence Summary

Freeze base, learn discrete k: efficient long-chunk online RL

ExRL keeps the actions in long chunks ($H \approx 50$) intact: it freezes the base policy and trains a lightweight off-policy RL adapter to decide **how many first k steps of each predicted chunk to execute before replanning**. Optimization shrinks from high-dim. continuous actions that grow with chunk length to picking one number in $\{1,\ldots,H\}$; DP, ACT, π0/π0.5, π0-FAST and Cosmos Policy all improve sample-efficiently.

{1, …, H}

Adaptation space: choose only the execution steps k per chunk; dimension does not grow with chunk length

>90%

ExRL-SAC success rate on DP Tool Hang (H=50); other baselines ~20% or lower

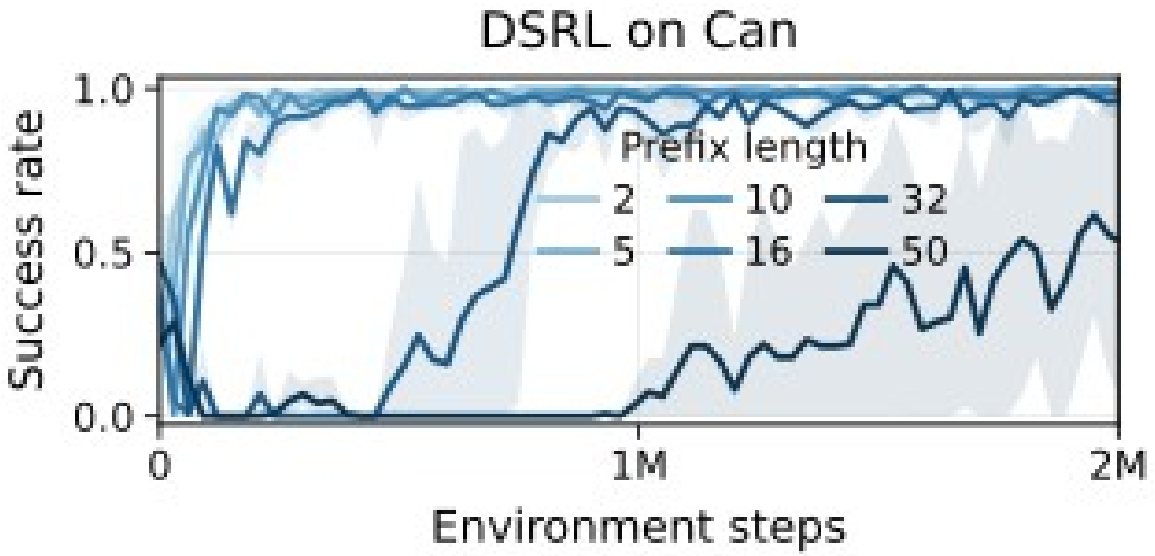
≈10 min

Physical interaction time for real-robot π0.5 block stacking; final eval 17/20 (9/20 with fixed k=10)

Longer chunks make action-space RL more sample-hungry

- Long chunks are mainstream: π0.5, π0-FAST, DreamZero etc. predict 20–50 steps executed open-loop, enabling 50 Hz control with fewer inference pauses
- Post-deployment online RL is still needed, but real-robot interaction is slow, costly, risky → sample efficiency is paramount
- Existing methods' optimization dim. grows linearly in H: residual RL learns $H \times d_a$ corrections, DSRL searches an $H \times d_a$ noise space, OGPO / PA-RL alter the whole generation process

Longer chunks → higher-dim. optimization → more online samples

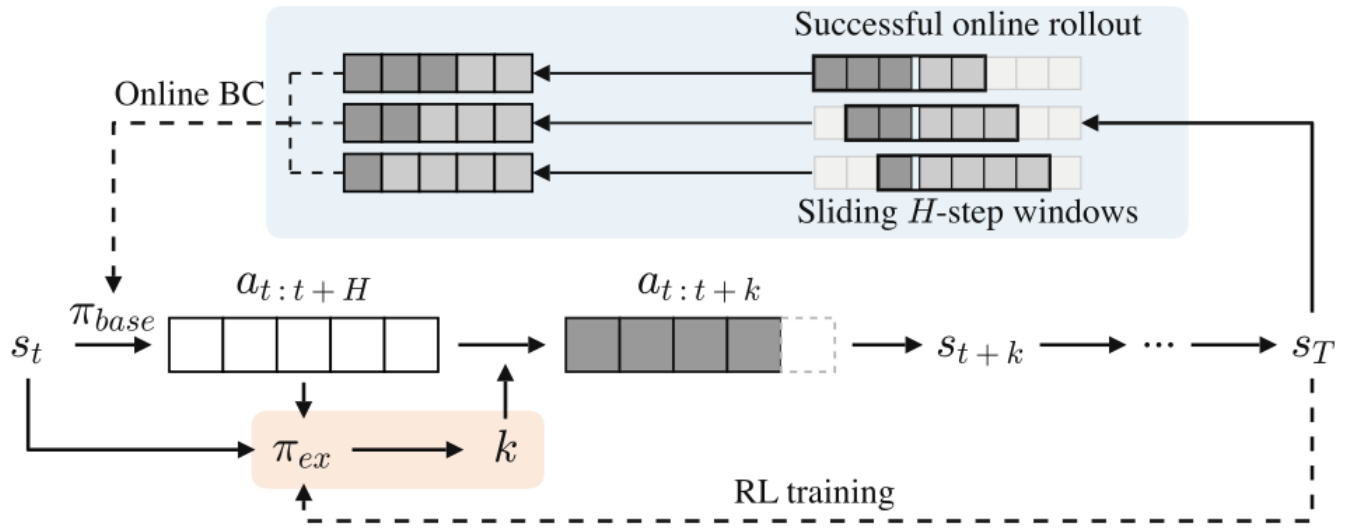


Controlled study: same DP on Can, exposing only the first L steps to DSRL. L=2–16 quickly reaches ~100%; L=32 is clearly slower; L=50 stays ~50% after 2M steps. Base success on Can is insensitive to L, so the gap stems mainly from optimization dimension.

Source: Sec. I; Fig. 11 (Appendix A-C, DSRL on Can)

Core Idea

New knob: keep actions, learn only steps run per chunk



Frozen π_base generates chunks; π_ex(k | s, chunk) only decides how many steps to execute before re-querying; optionally, successful trajectories are fed back into the base via Online BC.

Large k → long open-loop runs

Fully exploits chunks' temporal consistency; fewer inference pauses, smoother motion

Small k → timely replanning

Faster response to unexpected state changes and dynamics mismatch

Longer chunks suit it better

Choices of k grow with H → more expressivity; optimization stays a low-dim. discrete "pick 1 of H" problem

Source: Fig. 2; Sec. I, IV

Method • Value Learning

Duration-aware Q-learning: execution length as an SMDP “meta-action”

$$y_t = \sum_{i=0\ldots k-1} \gamma^i r(s_{t+i}, a_{t+i}) + \gamma^k \cdot V(s', a')$$

The critic is Q(s, a_{t:t+H}, k): reward accumulation and bootstrap discount both depend on the chosen k; a′ is resampled by the frozen base at new state s′. Appendix E proves this equals the semi-Markov decision process (SMDP) induced by the frozen base.

ExRL-Q: purely value-based

- $V(s',a') = \max_{k'} Q(s',a',k')$; at execution, k = argmax Q
- No actor, simplest; default in real-robot experiments
- CQL coefficient 0.3, UTD 1/4

ExRL-SAC: discrete actor-critic

- Actor outputs a categorical distribution over k; soft value sums exactly over all k
- Entropy term sustains exploration, preventing early collapse to a fixed k
- CQL coefficient 0.01, target entropy 0.98 · log H

Shared design: critic input = state + predicted chunk · ensemble of 10 critics · CQL curbs overestimation of poorly covered k · most hyperparameters shared across tasks

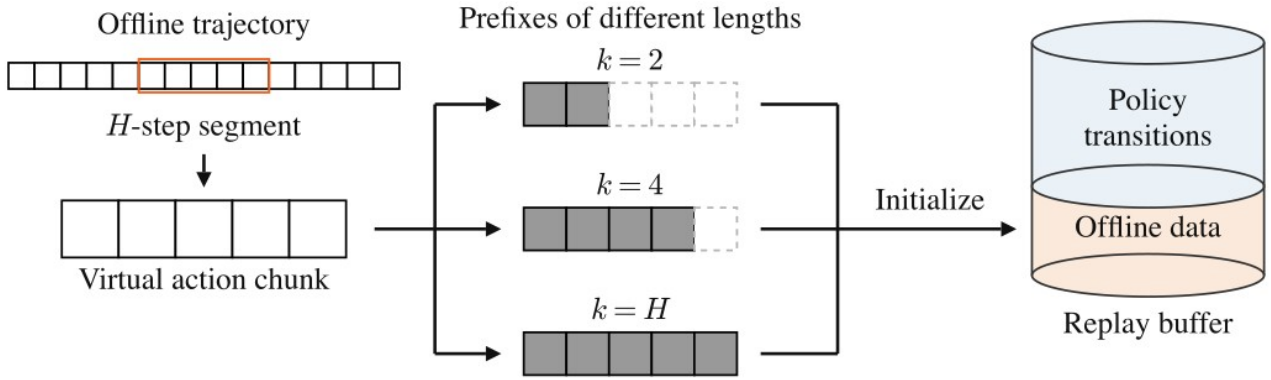
Source: Sec. IV, Eq. (3)–(9); Table III; Appendix E

Add-ons: distill to base; offline data → length data

① Online BC: beyond the base ceiling

- With a frozen base, ExRL can only select and stitch from the base's existing predictions
- Adaptive truncation = stitching prefixes of different chunks into new trajectories, yielding successes open-loop execution never produces
- Slice H-step sliding windows from successful rollouts as chunk targets; update π_base with the same BC loss as in pretraining
- DP / ACT add a frozen-teacher constraint to limit drift from pretrained behavior

② Offline data → execution-length experience

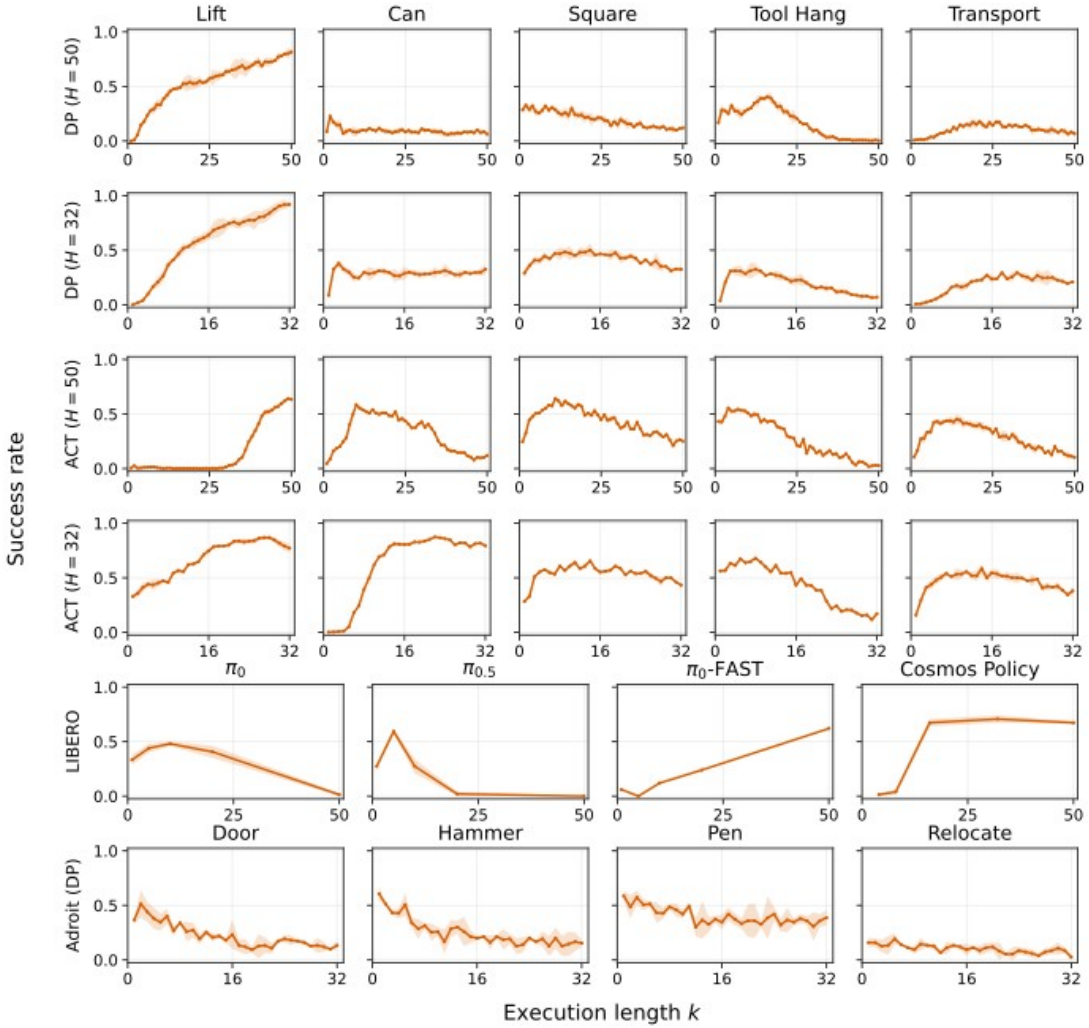


- Any H-step segment of an offline trajectory (even if not collected in chunks) = a virtual chunk; each of its length-k prefixes is genuine execution experience
- Bootstrapping uses the current base + current execution policy, anchoring values to ExRL's own behavior, not the demonstrator's

Source: Fig. 2, Fig. 3; Sec. IV

Why “Learn” k

The best fixed k varies by task and policy, and sweeping it is itself costly



Fixed-k sweep (frozen base, approx. from plots)

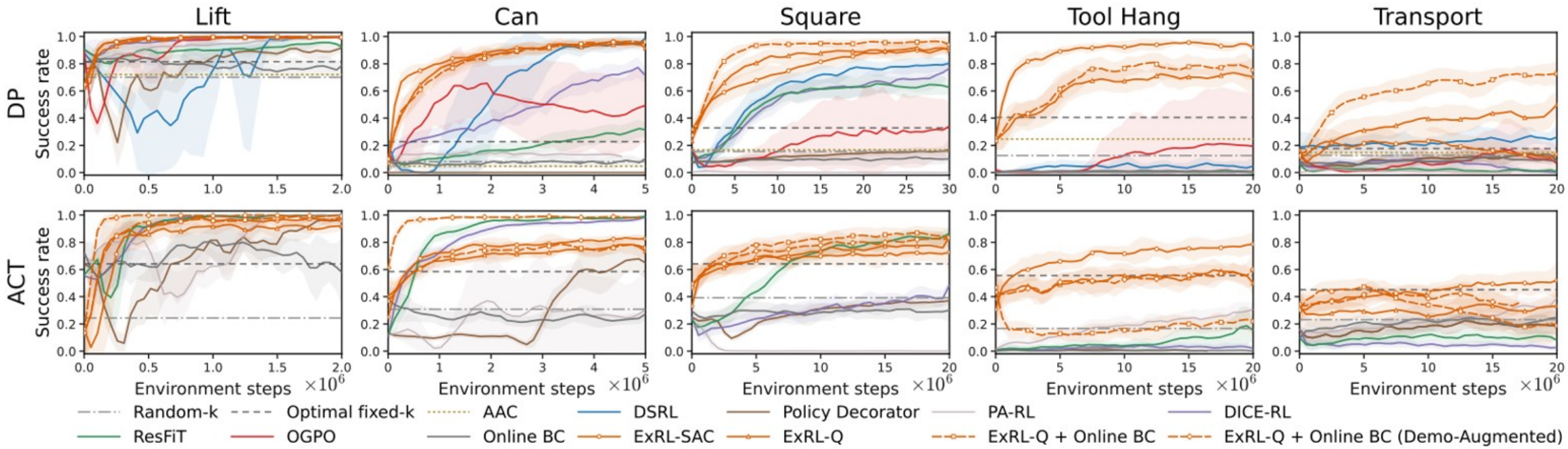
DP Lift （H=50）	Larger k is better; ≈0.8 at k=50
ACT Can / Square （H=50）	Peak ≈0.6 at k≈10
π0.5 • LIBERO	Peak ≈0.6 at k≈5; ≈0 if longer
π0-FAST • LIBERO	Rises with k; ≈0.6 at k=50
Adroit （DP）	Small k better overall

Plain RL with short k everywhere: initial performance may be very poor, inference is costly, and decision steps explode on long-horizon tasks.

Sweeping for the best fixed k first: needs hundreds of real-robot rollouts, nearly an entire online RL budget.

Source: Fig. 16 (Appendix C); Sec. IV Remark

Robomimic (H=50): learning only k yields large leads on hard tasks



>90% vs ⩽~20%

DP Tool Hang: ExRL-SAC vs. others

Beats best fixed k

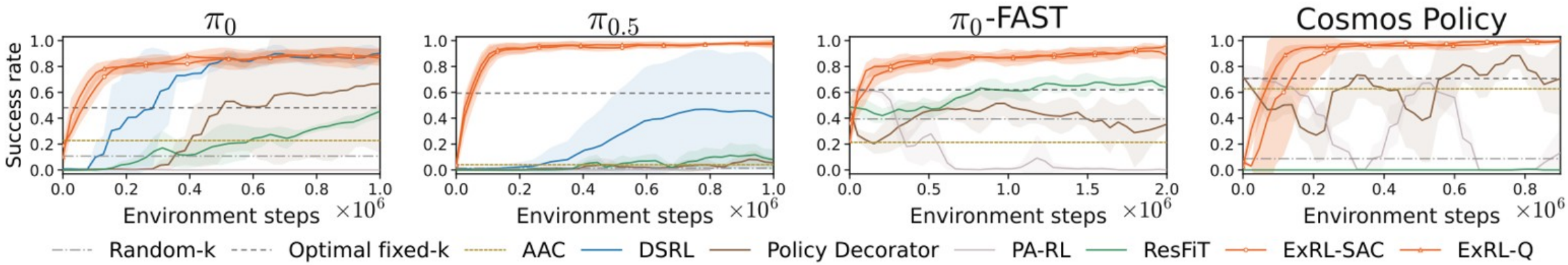
Largest gains on DP Can / Square / Tool Hang, with no base actions changed

Transport is hardest

Frozen base falls short; clear gains need ExRL-Q + Online BC (+ demos)

Source: Fig. 4 (DP / ACT, H=50, mean and 95% CI over 3 seeds)

Arch.-agnostic: flow/AR VLAs & world action models hit ~90%+



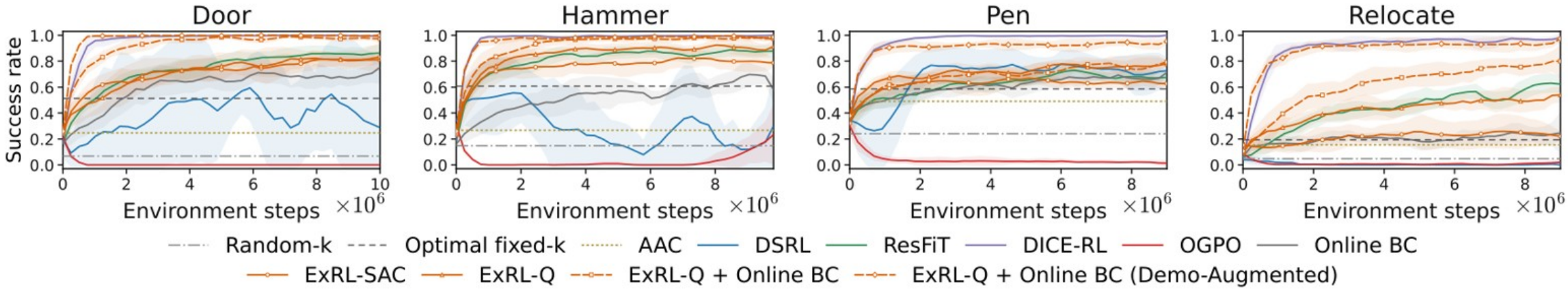
≈0% → >90%

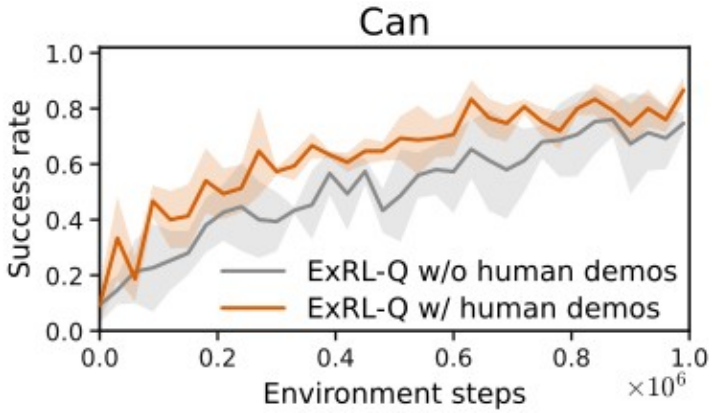
π0.5 within 0.2M env steps (LIBERO-90, single task)

- Four bases: flow VLAs (π0, π0.5), autoregressive VLA (π0-FAST), world action model (Cosmos Policy); all with chunk length 50
- One execution-length formulation adapts them all, without access to gradients, latents or the denoising process
- On π0.5: best fixed k ≈0.6, DSRL peaks at ≈0.4, other RL baselines near 0 (read from plot)

Source: Fig. 6 (LIBERO, H=50)

Online BC plus human demos breaks the “length-selection only” ceiling

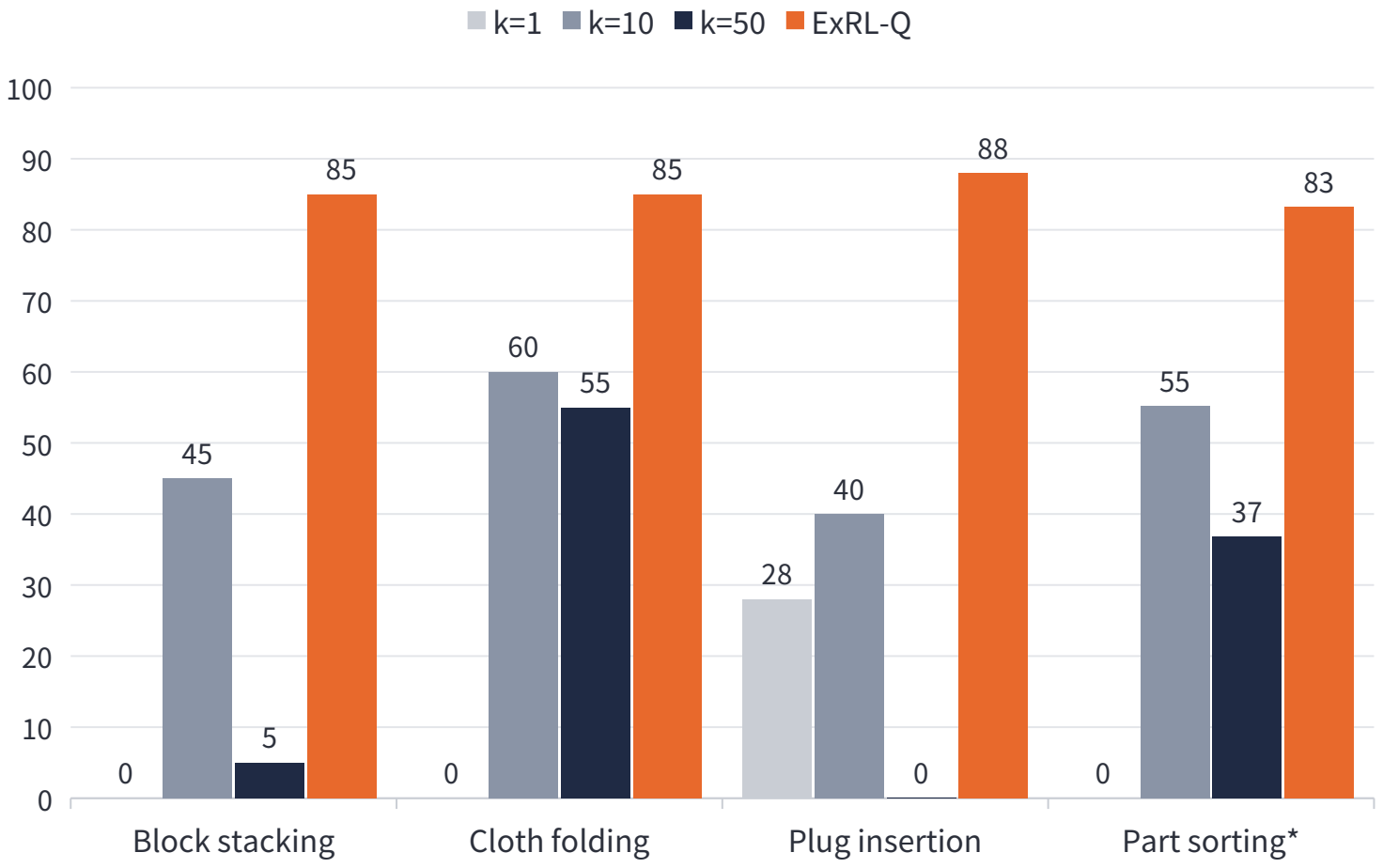




- Adroit (dexterous hand, H=32): frozen-base ExRL is weak on Relocate; Online BC helps clearly; adding demos brings all four tasks near 100%, on par with demo-augmented methods such as DICE-RL
- Robomimic: Online BC is especially effective on DP Transport; ExRL-Q + Online BC clearly beats ExRL-Q alone or Online BC alone
- Left (Can, H=50): adding human demos to the RL replay buffer, base still frozen, speeds up early learning

Source: Adroit learning curves (Sec. V-C); Fig. 7

Real π0.5 (frozen) clearly beats fixed k on 4/4 tasks



Success % (Table I: 17/20, 17/20, 22/25, 104/125); *Part sorting: completed / total stages

Power insertion task

Parts organization task

~10 min

Stacking ≈100%

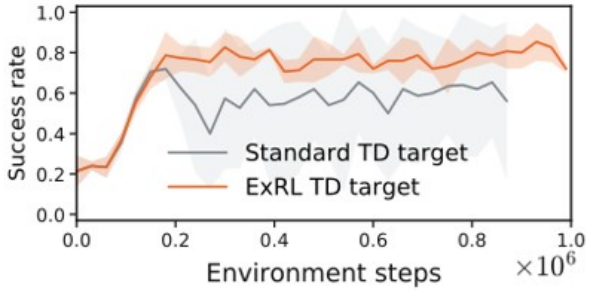
~85 min

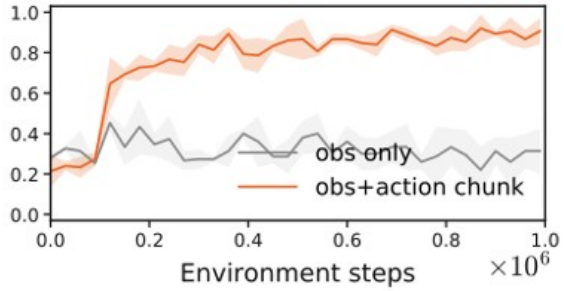
Cloth folding ≈90%

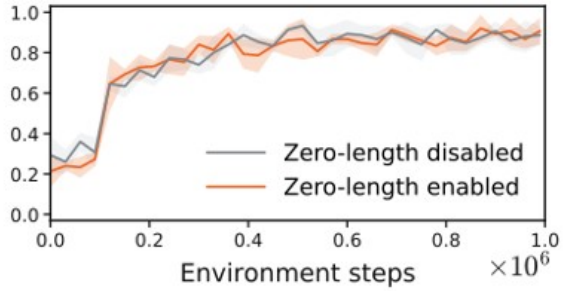
- AgileX Piper single arm, 20 Hz, H=50; only the execution-length critic is updated
- Plug insertion, part sorting: gains within a few hundred rollouts
- Later, rewards and discounts use physical time (incl. inference pauses) to curb excessive replanning

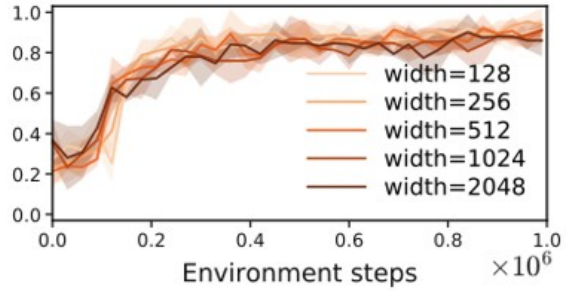
Source: Table I; Fig. 12; Appendix A-G

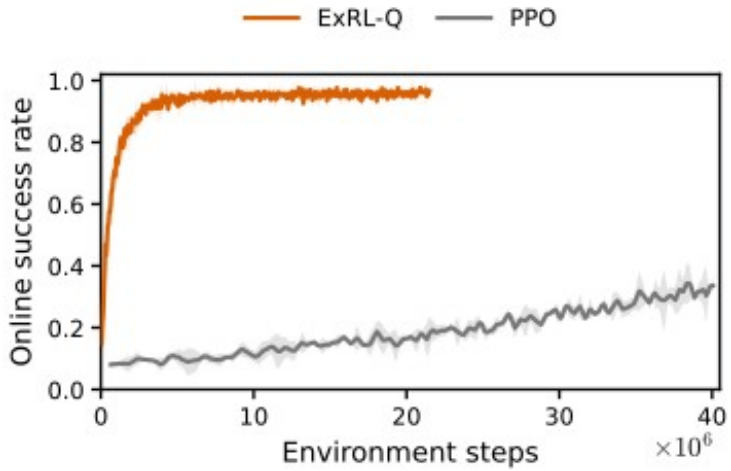
All help: duration-aware TD, chunk-aware critic, off-policy







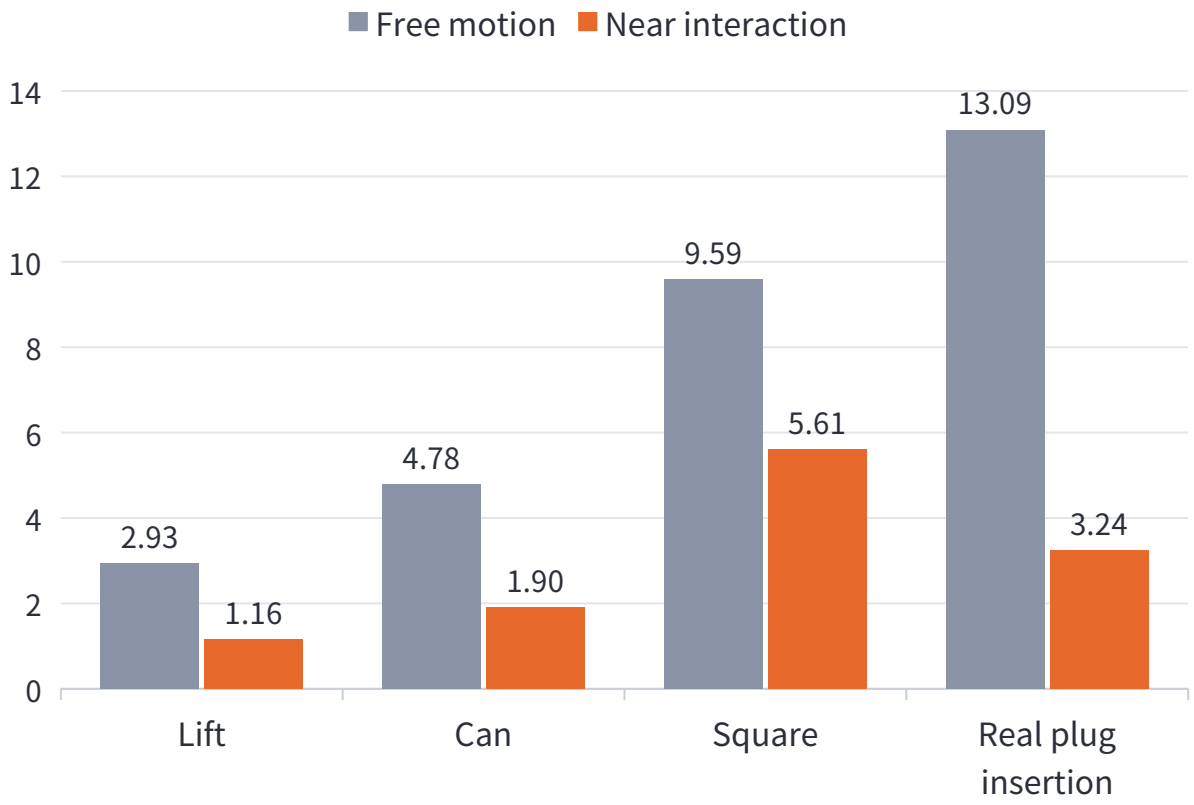




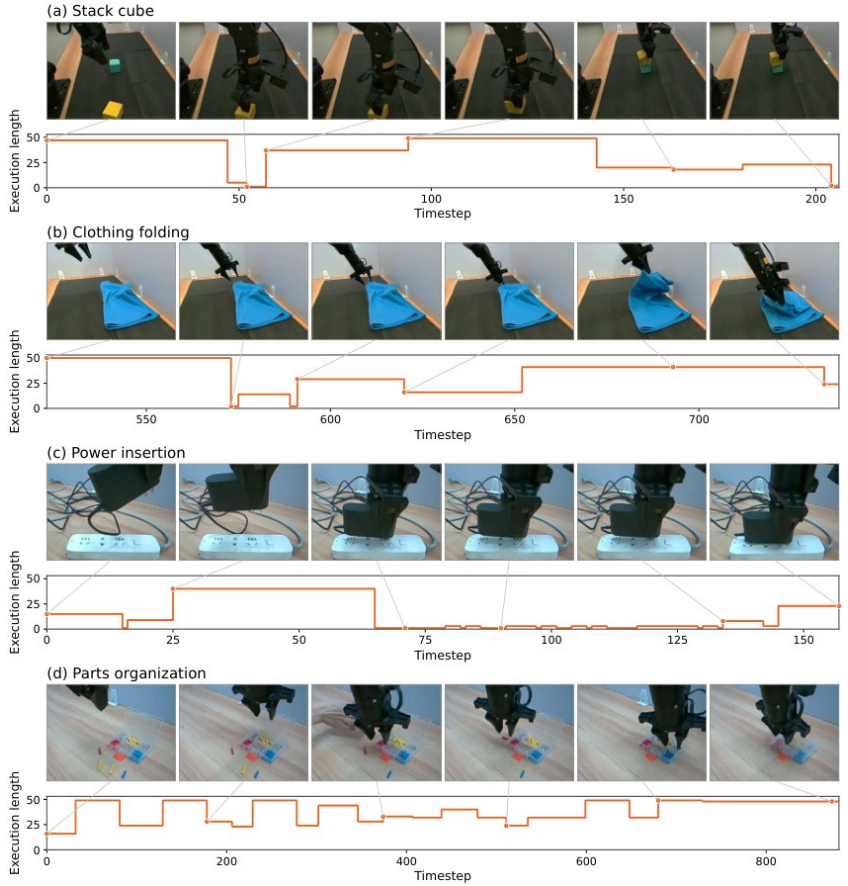
Duration-aware TD target	≈0.8 vs. ≈0.6 for one-step standard TD, which is also unstable
Critic sees predicted chunk	≈90% vs. ≈30% with observation only
Allowing k=0 / width 128–2048	Almost no difference → no need to tune network capacity
Off-policy vs. PPO (left)	ExRL-Q >90% in ~3M steps; PPO still <40% at 40M

Source: Fig. 8 (ablations); Fig. 14 (Appendix B, Can, H=50)

Learned policy: shorter execution near interaction, longer in free motion



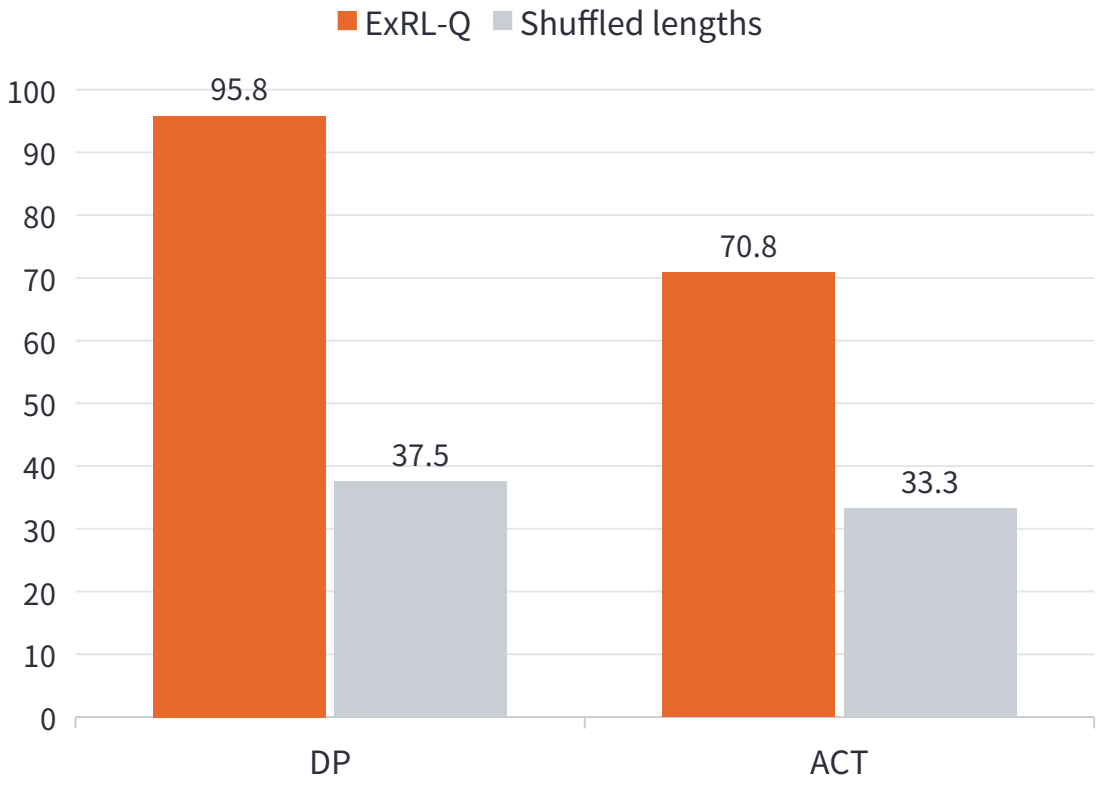
Mean execution length k (Fig. 9a). k shortens near interaction in all 9 simulated "task × seed" combinations and on the real robot; conclusions hold for 6 / 8 / 10 cm distance thresholds.



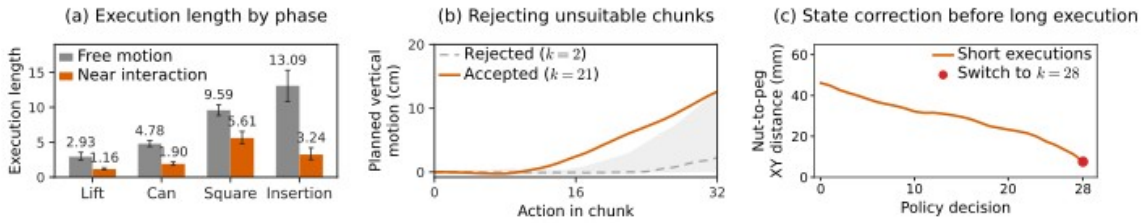
Real robot: block stacking / cloth folding use short k near grasps, long k in transport; plug insertion keeps short k in contact; part sorting stays long throughout.

Source: Fig. 9(a); Fig. 17 (Appendix D-A)

Key is “which k when”: shuffling cuts success 38–58 pp



Square (H=32) success %, 24 paired rollouts per group (shared initial state, initial chunk and inference randomness). Gain (95% CI): DP 58.3 [37.5, 79.2], ACT 37.5 [16.7, 58.3].



Short k ①: veto bad plans

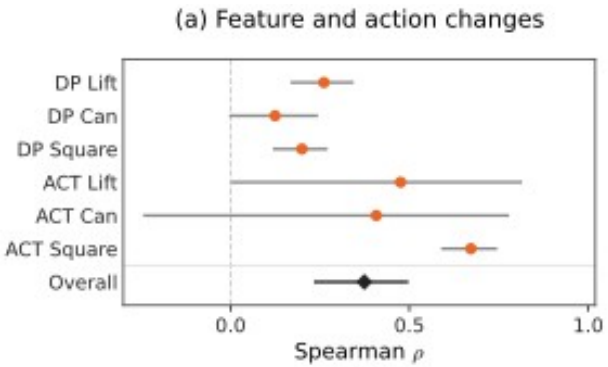
On Square, repeatedly truncates near-static chunks at k=2 until one plans a ~12 cm grasp-and-lift, then executes it with k=21 (Fig. b)

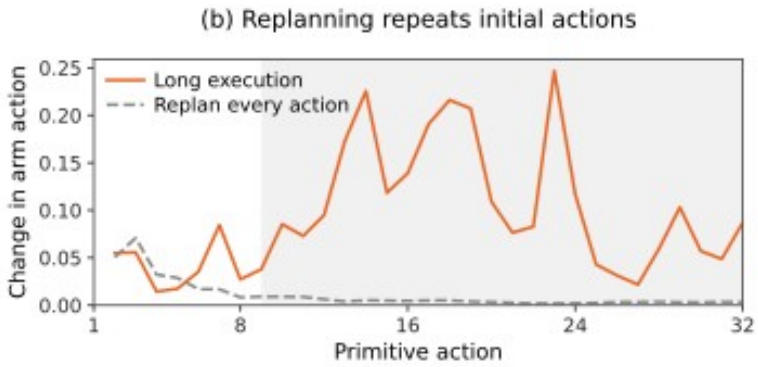
Short k ②: small fixes

Successive short executions shrink the nut-to-peg XY distance from ~45 mm to ~8 mm, then switch to k=28 to finish (Fig. c)

Source: Table VIII (Appendix D-B); Fig. 9(b)(c)

Always short k? Aliasing “restarts” the same action every replan





ρ = 0.374

Spearman corr. of feature change vs. new first-action change (95% CI [0.237, 0.494]); positive for all DP / ACT × 3 tasks

3.6%

ACT Square: per-step replanning moves EE 19.4 mm; command change is 3.6% of the original chunk's

From 3 selected ACT Square states (Table IX; diagnostic examples, not benchmark success rates):

Always k=1 → 1/3 • k=32 first, then ExRL-Q → 1/3 • ExRL-Q throughout → **2/3**

Insight: balancing “needing a new prediction” against “letting the current one unfold” is beyond any fixed policy.

Source: Fig. 18; Table IX (Appendix D-C)

Assessment

Practical strengths, but mind the scope of the conclusions

Strengths

- Simple angle: expressivity grows with H while optimization dim. does not, tightly linked to the Fig. 11 evidence
- Truly architecture-agnostic: only needs a chunk-outputting base; suits closed-source / huge VLAs and world models
- Engineering-friendly: ExRL-Q has no actor, is insensitive to network width, hyperparameters mostly shared; real robot needs only ~10–85 min of interaction
- Solid analysis: shuffling controls and feature aliasing explain “why changing only length suffices”

Limitations & Questions

- Frozen base caps performance; no Online BC on the real robot, some tasks still below 100% (authors' note)
- Setup favors long chunks: many baselines were designed for short chunks; the gap narrows markedly at H=32
- Base checkpoints deliberately leave headroom (e.g., ACT Lift trained for only 0.12 epochs)
- Short k means more large-model inference; compute cost in simulation not compared separately
- Single-task training; multi-task generalization and combination with async inference such as RTC untested (authors' note)

Source: Sec. VII Limitations; Table II; Fig. 13; reviewer's assessment

Three Takeaways

01

“When to replan” is an underrated control variable

For long-chunk policies, tuning only execution length unlocks much existing base capability

02

Low-dim. discrete interface + off-policy RL

Brings online sample needs to real-robot scale: tens of minutes or a few hundred rollouts

03

Practical recipe: tune length first, then distill

Cheap, weight-free gains via ExRL; Online BC then writes found behaviors to the base